

Contents

1	plans	1
1.1	week 1: introduction	1
1.1.1	introductions, 15	1
1.1.2	syllabus review, 20	1
1.1.3	HAI article, 40-45	2
1.1.4	break	2
1.1.5	python challenge, 30	2
1.1.6	share, 20-30	3
1.1.7	python refresher (if time)	3
1.2	week 2: data & bias	3
1.2.1	provocations sharing 30 minutes	3
1.2.2	cleaning a dataset	4
1.2.3	break	7
1.2.4	continue cleaning	7
1.2.5	start homework	7
1.3	week 3:	7

1 plans

1.1 week 1: introduction

1.1.1 introductions, 15

- your name, program, pronouns, last thing you watched in transit

1.1.2 syllabus review, 20

2 main goals of the course:

- traditional ML education:
 - learn the most important algorithms (linear, logistic, also unsupervised)
 - learn some principles about model architecture
- less traditional:
 - examine the most current ML research for common issues.
 - problems with bias, automation, sustainability, dataset curation, etc.

- considering the ways that coding tools provide much of the algorithmic knowledge, this expertise with critical skills will give you an edge when applying for ML jobs, or when using ML tools in your job.

The main assignment of the course is actually a paper, a conference paper. Requires you to know and respond to the most current research.

1.1.3 HAI article, 40-45

- Read the article & answer prompts, 15 minutes
 - 10 minutes read, 5 minutes answer
 - prompts for reading:
 - * In your understanding, why do these automated processes discriminate, what causes them to discriminate? Try to narrow down the root cause.
 - * Do you perceive any potential oversights or flaws in the authors' arguments? Is there anything that should be interrogated or explored in more depth?
- Discuss in small groups, 10 minutes.
 - introduce yourself to your group
- Discuss as large group, 15 minutes
 - root cause: that increasing scale brings in all kinds of problems. The common denominator.q
 - oversight: that if you apply to a lot of jobs, you're maybe using a similar application, which is not tailored and may be rejected for not being a good fit.

1.1.4 break

1.1.5 python challenge, 30

Write some python code that contains the following components below. Use a blank sheet of paper to code.

Required:

- a loop

- a list
- a function

One solution:

```
import random
def greeting(name):
    greetings = [
        "how are you doing today?",
        "what's up?",
        "como te sientes?",
        "what's shakin'?",
        "tudo bem?"
    ]
    print(f"Hello {name}, {random.choice(greetings)}")
```

```
greeting('Filipa')
```

Hello Filipa, como te sientes?

1.1.6 share, 20-30

1.1.7 python refresher (if time)

- data types
- variables
- list indexing, slicing
- loops
- logic
- functions
- dictionaries
- libraries

1.2 week 2: data & bias

1.2.1 provocations sharing 30 minutes

- in partners, share your provocations. 5 minutes.
- share out with the class. 10 minutes.

1.2.2 cleaning a dataset

- knowing the elements & limitations of a dataset
- decide what you want to predict
- cleaning the dataset accordingly

1. preliminary exploration

```
import pandas as pd

df = pd.read_csv("2026-bills.csv")

df.shape
df.info()
df.head()
df.value_counts('Bill_Type')
df['State']
```

2. GROUPS: close reading dataset in groups, 10 minutes

- pass out the printout
- what kind of predictions could we make from the data?
 - think about which of these columns can be used to guess about another column.
 - * Example:
 - * what states are likely to have bills become "enacted"?
 - * `enacted = df['Manual Status'] == 'Enacted'`
 - how to label a bill based on its summary and notes.
 - * Summary + Notes = Bill Type
 - other things which could be interesting, but we won't pursue (for now):
 - * Enacted - what states/sponsors/types are likely to be enacted?
 - * Sponsors - are there any patterns in who is sponsoring certain bills?
 - * State - are there patterns in which states have certain bills?

3. example: "enacted"

Let's create a sub-df with just bills which are "enacted"

What kinds of questions can we ask about this data?

- what is the Bill Type that's likely to be enacted?

4. cleaning

- we need three columns: summary, notes, bill type
 - examine each column to look for inconsistencies
 - what do we need to clean from each column, if at all?

Start with Bill Type. What are the most frequent values for this column?

```
df["Bill Type"].value_counts()
```

There are 218 different labels. That's because we have examples like the one below. We need to cut these down.

```
"DEI Ban" "DEI Ban " " DEI Ban" "DEI Ban "
```

First, preserve the original.

```
df["Bill Type Raw"] = df["Bill Type"]
```

Then, let's split up the long string into individual ones.

```
df["Bill Type"].str.split("/")
```

Now, create a new column for the split labels.

```
df["bill_type_labels"] = (  
    df["Bill Type"]  
    .fillna("")  
    .str.split("/")  
)
```

Use `strip()` to remove white spaces.

```
def clean_labels(labels):
    return [label.strip() for label in labels]

df["bill_type_labels"] = df["bill_type_labels"].apply(clean_labels)
```

Make the difference between these two columns visible.

```
print(df["Bill Type Raw"].iloc[0])
print(df["bill_type_labels"].iloc[0])
```

Now, to pull out the individual lable, use `explode()`

```
labels = df["bill_type_labels"].explode()
```

We can ask how many times each label is used!

```
labels.value_counts()
labels.unique()
# see the comparison between the original and our new categories.
len(labels.unique())
len(df['Bill Type Raw'].unique())
```

Are there any other cleaning steps we'd want to do?

- remove labels that appear 1 time? less than 3 times?

5. final step — MultiLabelBinarizer To train a classifier, we would create a "sparse" matrix. A sparse matrix has 0s and 1s to indicate the presence of a category.

We will use the library `sklearn`.

```
from sklearn.preprocessing import MultiLabelBinarizer

mlb = MultiLabelBinarizer()

Y = mlb.fit_transform(df["bill_type_labels"])
```

This creates an `mlb` object and `Y` object. The first contains our classes, and the second is our matrix.

```

# all of our unique labels
mlb.classes_

# scary matrix...
Y

# see that we have the same number of rows as bills
# and we have the same number of columns as labels!!
Y.shape()

```

6. our workflow

RAW BILL TYPE -> Preserve original -> Split on "/" -> Clean
whitespace -> MultiLabelBinarizer -> 0 / 1 matrix

1.2.3 break

1.2.4 continue cleaning

1.2.5 start homework

1.3 week 3: